

運用 Gemini Code Assist 加速開發

來源：<https://codelabs.developers.google.com/genai-for-dev-gca-dev-use-cases?hl=zh-tw>

步驟數：12

運用 Gemini Code Assist 加速開發

目錄

1. 總覽
2. 設定和需求
3. 測試提示的選項
4. 下載並檢查應用程式
5. 在本機啟動應用程式
6. 新增說明文件
7. 新增單元測試
8. 檢查是否有錯誤
9. 重構 UI
10. 重構後端
11. Gemini Code Assist 結論
12. 恭喜！

步驟 1 · 約 0 分鐘

1. 總覽

本實驗室的重點是運用 Google Cloud 的 AI 輔助代理程式 Gemini Code Assist。您將學習如何使用 Gemini Code Assist 執行常見的開發人員工作，包括瞭解現有程式碼集、生成說明文件和單元測試，以及重構 Python 網頁應用程式的 UI 和後端元件。

學習目標

在本實驗室中，您將瞭解如何執行下列操作：

- 如何使用 Gemini Code Assist 處理常見的開發人員工作。

必要條件

- 本實驗室假設您已熟悉 Cloud 控制台和 Cloud Shell 環境。
-

步驟 2 · 約 0 分鐘

2. 設定和需求

設定 Cloud 專案

1. 登入 [Google Cloud 控制台](#)，然後建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。

The screenshot shows the Google Cloud console interface. At the top, there is a 'Select a project' dropdown menu. Below it, the 'Select a project' page is visible, featuring a search bar labeled 'Search projects and folders'. To the right of the search bar is a 'NEW PROJECT' button. Below the search bar, the 'New Project' form is shown. It includes a 'Project name' field with the text 'My Project 13475' and a question mark icon. Below the project name is the 'Project ID' field, which contains the text 'inbound-analogy-389614'. A note next to the Project ID states 'It cannot be changed later. EDIT'. Below the Project ID is the 'Location' field, which is empty, and a 'BROWSE' button. At the bottom of the form are 'CREATE' and 'CANCEL' buttons.

- **專案名稱**是這個專案參與者的顯示名稱。這是 Google API 未使用的字元字串。你隨時可以更新。
- **專案 ID** 在所有 Google Cloud 專案中都是不重複的，而且設定後即無法變更。Cloud 控制台會自動產生專屬字串，通常您不需要在意該字串為何。在大多數程式碼研究室中，您需要參照專案 ID (通常標示為 `PROJECT_ID`)。如果您不喜歡產生的 ID，可以產生另一個隨機 ID。你也可以嘗試使用自己的名稱，看看是否可用。完成這個步驟後就無法變更，且專案期間會維持不變。
- 請注意，有些 API 會使用第三個值，也就是「專案編號」。如要進一步瞭解這三種值，請參閱[說明文件](#)。

注意：專案 ID 在全域中是唯一的，選定後就無法再供他人使用。您是該 ID 的唯一使用者。即使專案已刪除，ID 也無法再次使用

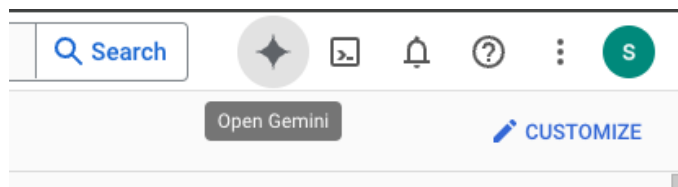
注意：如果使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果你使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

2. 接著，您需要在 Cloud 控制台中[啟用帳單](#)，才能使用 Cloud 資源/API。完成這個程式碼研究室的費用不高，甚至可能完全免費。如要關閉資源，避免在本教學課程結束後繼續產生費用，請刪除您建立的資源或專案。Google Cloud 新使

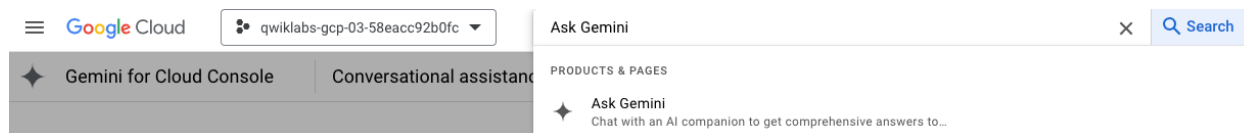
用戶可參加價值\$300 美元的免費試用計畫。

環境設定

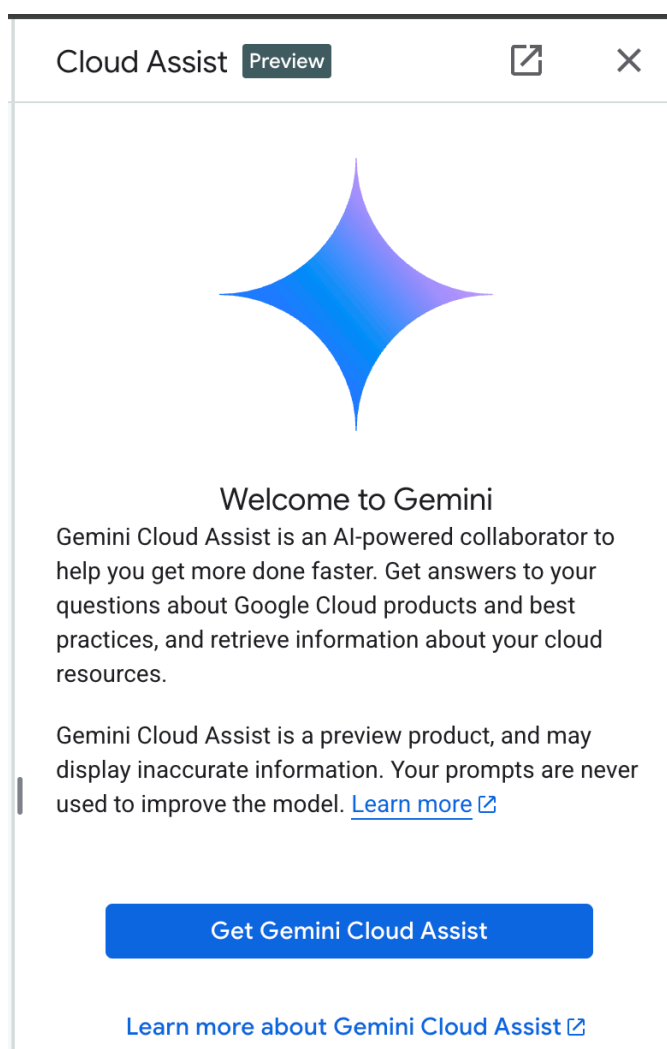
開啟 Gemini 對話。



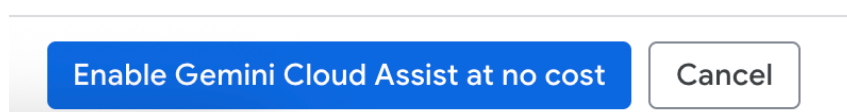
或在搜尋列中輸入「問問 Gemini」。



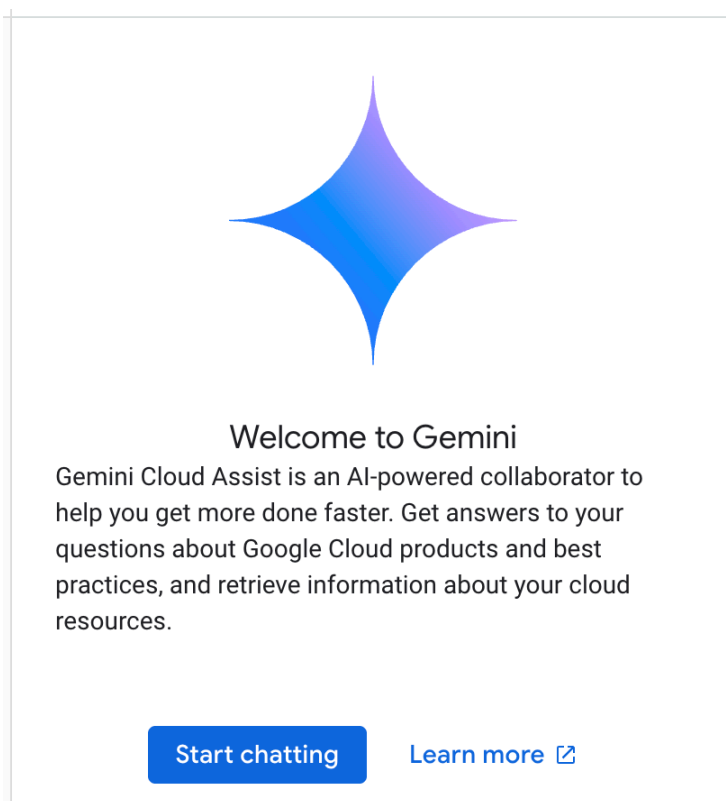
啟用 Gemini for Google Cloud API：



在下一個畫面中啟用 GCA。



按一下「`Start chatting`」，然後按照其中一個範例問題操作，或自行輸入提示詞來試用。



建議提示詞：

- 請用 5 個重點說明 Cloud Run。
- 您是 Google Cloud Run 產品經理，請用 5 個簡短重點向學生說明 Cloud Run。
- 您是 Google Cloud Run 產品經理，請用 5 個簡短重點向認證 Kubernetes 開發人員說明 Cloud Run。
- 您是 Google Cloud Run 產品經理，請以 5 個簡短重點，向資深開發人員說明何時該使用 Cloud Run，何時該使用 GKE。

完成後，請關閉 Gemini Cloud Assist 對話視窗。

如要進一步瞭解如何撰寫更優質的提示，請參閱[提示指南](#)。

瞭解 Gemini for Google Cloud 如何使用您的資料

Google 對隱私權的承諾

Google 是業界首批發布 [AI/機器學習隱私權承諾](#) 的公司，該文提到我們的信念：除了極致的安全性之外，客戶也應該對儲存在雲端的自家資料保有最大的掌控權。

您提交及收到的資料

您向 Gemini 提出的問題 (包括提交給 Gemini 分析或完成的任何輸入資訊或程式碼)，都稱為提示。您從 Gemini 取得的答案或程式碼完成建議稱為「回覆」。我們不會將您的提示詞或 Gemini 的回覆用做模型訓練資料。

提示加密

當您將提示提交給 Gemini 時，您的資料會在傳輸過程中加密，然後輸入 Gemini 的基礎模型。

Gemini 生成的節目資料

Gemini 是以 Google Cloud 第一方程式碼和精選第三方程式碼訓練而成。您必須對程式碼的安全性、測試和效用負責，包括 Gemini 提供的任何程式碼完成、生成或分析功能。

[進一步瞭解](#) Google 如何處理提示。

步驟 3 · 約 0 分鐘

3. 測試提示的選項

如要變更現有提示，可以採取下列做法。

- [Vertex AI Studio](#)

Vertex AI Studio 是 Google Cloud Vertex AI 平台的一部分，專門用於簡化及加速生成式 AI 模型的開發和使用。

- [Google AI Studio](#)

Google AI Studio 是網頁工具，可讓您設計提示工程原型，並透過 Gemini API 進行實驗。

- [Gemini 網頁應用程式](#) (gemini.google.com)

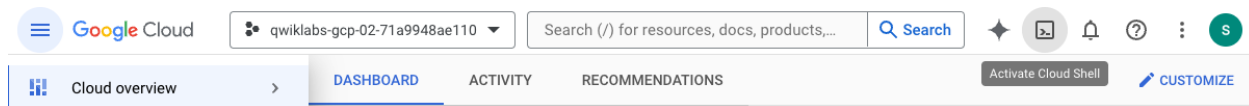
Google Gemini 網頁應用程式 (gemini.google.com) 是一款網頁工具，可協助您探索及運用 Google Gemini AI 模型強大的功能。

- [Android](#)和 [iOS 版 Google 應用程式](#)
-

步驟 4 · 約 0 分鐘

4. 下載並檢查應用程式

按一下搜尋列右側的圖示，啟用 Cloud Shell。



如果系統提示您授權，請點選「授權」繼續操作。

Authorize Cloud Shell

gcloud is requesting your credentials to make a GCP API call.

Click to authorize this and future calls that require your credentials.

REJECT

AUTHORIZE

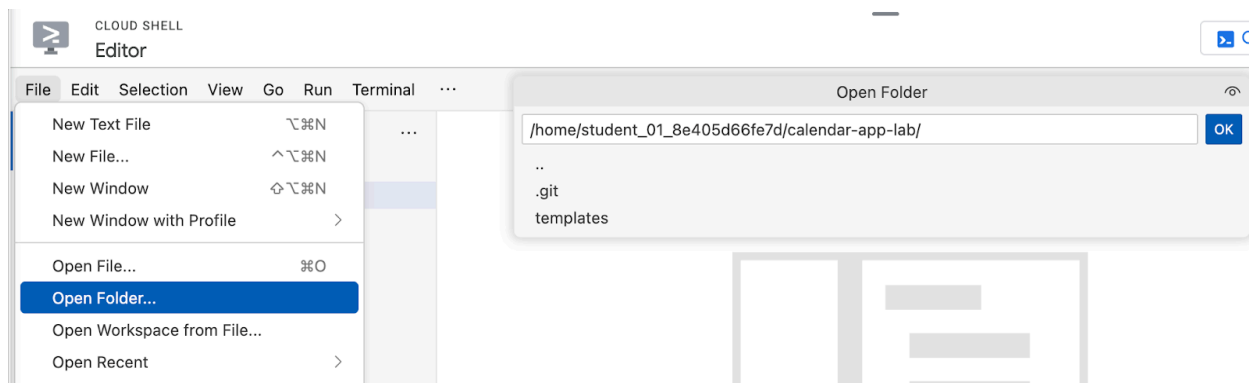
在終端機中執行下列指令，在本機複製 Git 存放區。

```
git clone https://github.com/gitrey/calendar-app-lab
cd calendar-app-lab
```

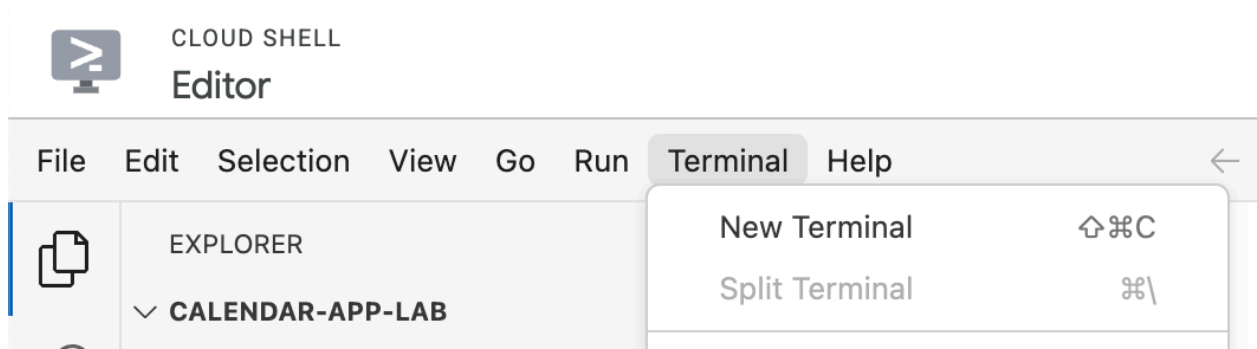
啟動「Cloud Shell 編輯器」。



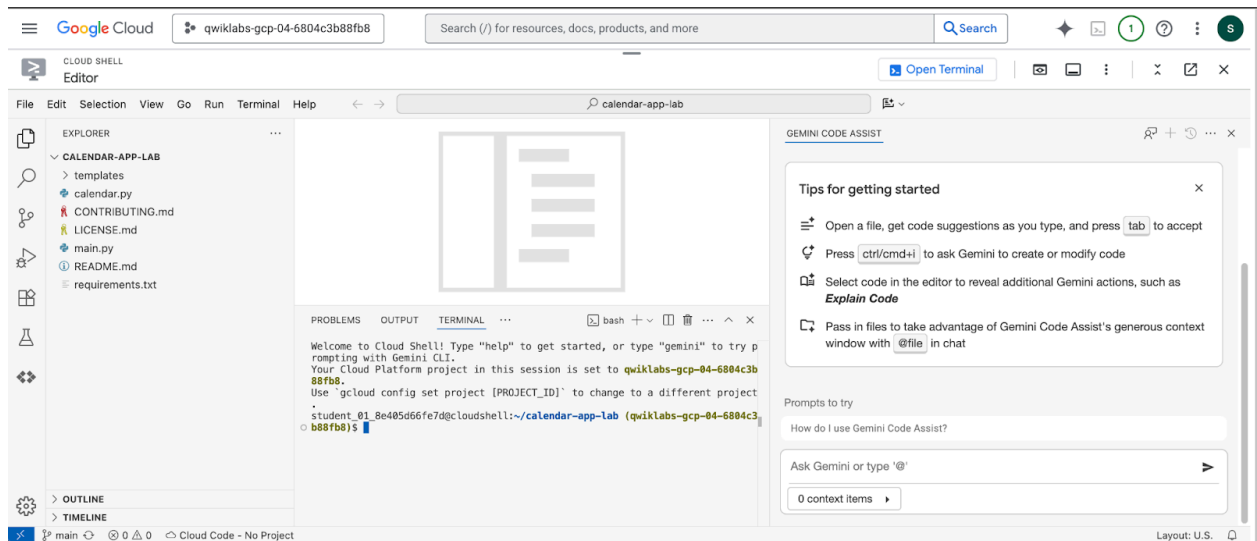
開啟「calendar-app-lab」資料夾。



在 Cloud Shell 編輯器中啟動新的終端機。



您的環境應如下方螢幕截圖所示。



在 Gemini Code Assist 對話視窗中，傳送下列提示：

Don't suggest any changes. Explain this codebase to me.

輸出內容範例：

Gemini

Thinking

Of course! I can explain the `calendar-app-lab` application to you.

Based on the files provided, `calendar-app-lab` is a simple web application built using the Python Flask framework. Its purpose is to convert a given number into its Roman numeral representation.

How it Works

The application has a straightforward user flow:

1. A user visits the home page, which displays a simple form.
2. The user enters a number into the input field and clicks the "Convert!" button.
3. The application processes this number, converts it to a Roman numeral, and displays the result on a new page.

Key Files

The application's functionality is split across a few key files:

1. `main.py`: This is the heart of the web application. It uses Flask to define two web pages (routes):
 - The home page (`/`), which shows the initial form (`templates/index.html`).
 - The conversion result page (`/convert`), which receives the number from the form, calls the conversion logic, and displays the result (`templates/convert.html`).
2. `calendar.py`: This file contains the core logic for the conversion. The `number_to_roman` function takes a number and iteratively builds the Roman numeral string.

Prompts to try

The roman numeral conversion is incorrect for the number 4. Can you fix it?

Write a unit test for the `number_to_roman` function.

Ask Gemini or type '@'



1 context item ▶

步驟 5 · 約 0 分鐘

5. 在本機啟動應用程式

在對話視窗中傳送下列提示：

How do I set up a virtual environment and run this app locally?

在終端機中執行建議的指令：

Running the Application

1. **Create and activate a virtual environment (Recommended):** It's a best practice to create a virtual environment to manage project dependencies. Open your terminal in the `calendar-app-lab` directory and run:

```
bash

# Create a virtual environment named 'venv'
python3 -m venv venv
```

Run in terminal

安裝依附元件：

2. **Install the required packages:** The `requirements.txt` file lists all the Python libraries the app needs. Install them using `pip`:

```
bash

pip install -r requirements.txt
```

Run in terminal

啟動應用程式：

3. **Run the Flask application:** Execute the `main.py` file to start the development server:

```
bash

python3 main.py
```

Run in terminal

按一下連結即可預覽應用程式：

```
PROBLEMS  TERMINAL  ...  python3  +  -  [ ]  [X]  ...  ^  X

Successfully installed Jinja2-3.1.6 MarkupSafe-3.0.2 Werkzeug-3
.1.3 blinker-1.9.0 click-8.2.1 flask-3.0.2 itsdangerous-2.2.0
(venv) student_01_8e405d66fe7d@cloudshell:~/calendar-app-lab (q
wiklabs-gcp-04-6804c3b88fb8)$ python3 main.py
* Serving Flask app 'main'
* Debug mode: on
WARNING: This is a development server. Do not use it in a produ
ction deployment. Follow link (cmd + click) to run in a produ
ction deployment.
* Running on http://127.0.0.1:8080
* Running on http://10.88.0.3:8080
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 840-509-626
```

輸出內容範例：

Roman Numerals

Enter a number:

Convert!

Result

The number 25 is XXV in Roman Numerals.

[Return to home page.](#)

步驟 6 · 約 0 分鐘

6. 新增說明文件

在對話視窗中傳送下列提示：

```
Add docstrings to all files
```

查看建議的變更並在對話中接受：

GEMINI CODE ASSIST

+

↺

...

×

Add docstrings to all files

 Gemini

⬆

 Thinking

Of course! Adding docstrings is a great way to improve code documentation and maintainability. As a world-class software engineering assistant, I can certainly help with that.

Here are the changes with docstrings added to your Python files.

In `calendar.py`, I'll add a module-level docstring and one for the `number_to_roman` function to explain its purpose, arguments, and return value.

⬆

`calendar.py` -0 +13

📄

⌕

✓

Next, in `main.py`, I'll add docstrings for the module and for each of the Flask route functions.

⬆

`main.py` -0 +12

📄

⌕

✓

```
# See the License for the specific language governing permissions and
# limitations under the License.

"""This module implements a simple Flask web application to convert numbers to Roman numerals."""

import os
```

Accept changes

Prompts to try

The roman numeral conversion is incorrect for the number 4. Can you fix it?

Write a unit test for the `number_to_roman` function.

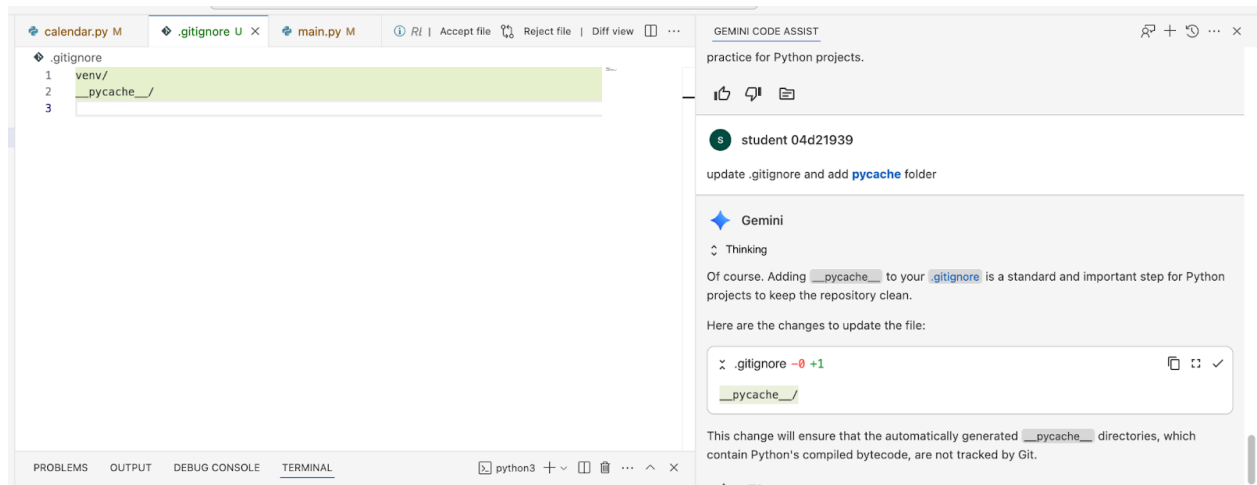
在對話視窗中傳送下列提示：

```
update .gitignore and add venv/* folder
```

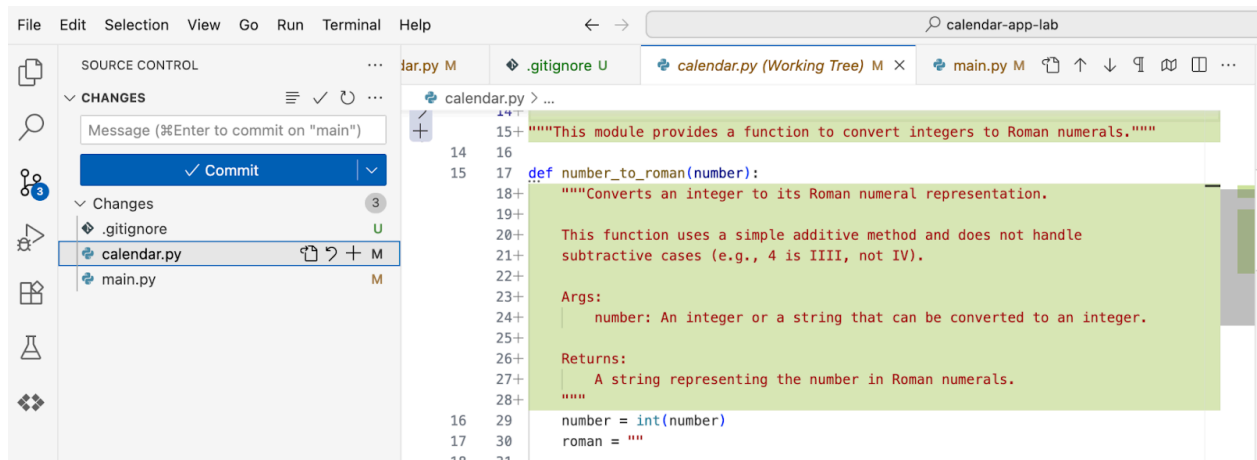
然後輸入以下提示：

```
update .gitignore and add __pycache__ folder
```

輸出內容範例：



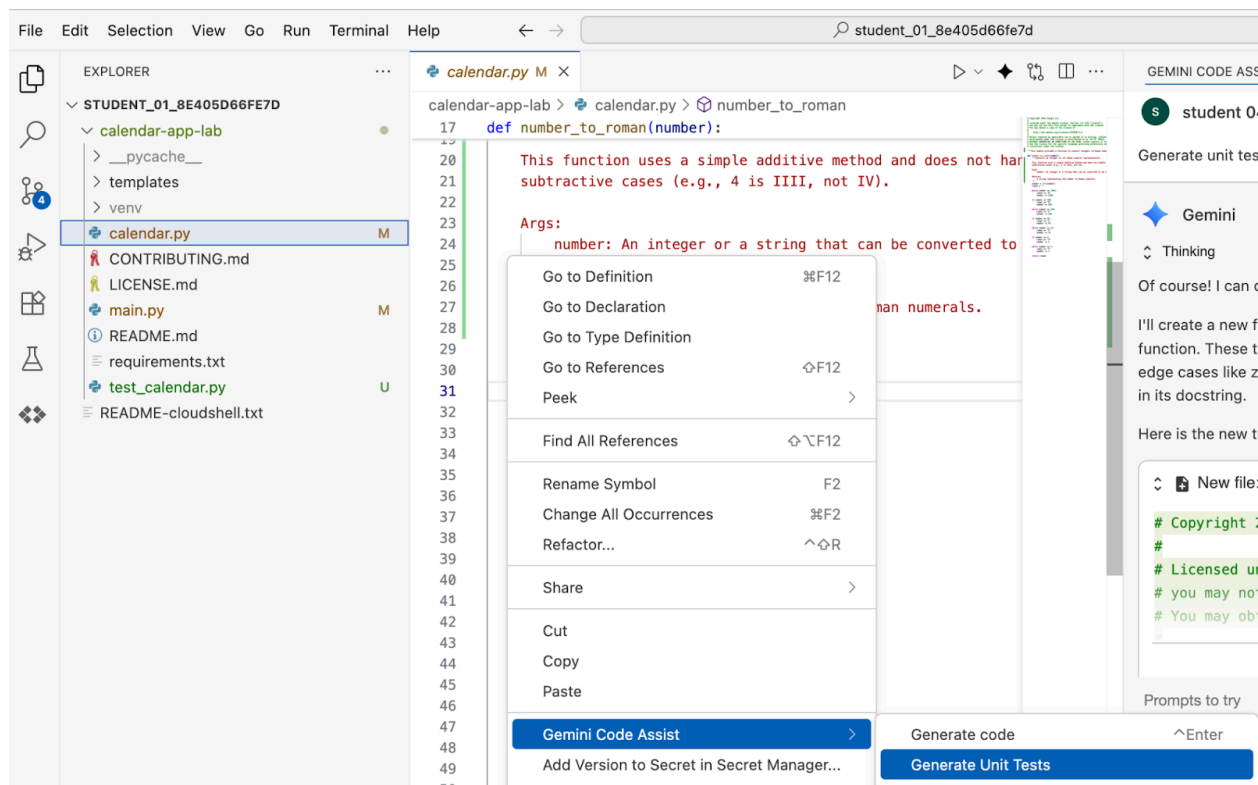
切換至「原始碼控管」檢視畫面，查看目前所做的變更：



步驟 7 · 約 0 分鐘

7. 新增單元測試

開啟 `calendar.py` 檔案，然後從內容選單選取 `Gemini Code Assist >> Generate Unit Tests`



在 Gemini Code Assist 對話視窗中按下 Enter 鍵。查看並接受變更。

student 04d21939

Generate unit tests

Gemini

⌵ Thinking

Of course! I can certainly generate unit tests for the `calendar.py` file.

I'll create a new file, `test_calendar.py`, which will contain a suite of tests for the `number_to_roman` function. These tests will cover various scenarios, including simple numbers, complex numbers, edge cases like zero, and how it handles different input types, confirming the behavior described in its docstring.

Here is the new test file:

⌵ New file: test_calendar.py +74

```
# Copyright 2024 Google LLC
#
# Licensed under the Apache License, Version 2.0 (the "License");
# you may not use this file except in compliance with the License.
# You may obtain a copy of the License at
```

Accept changes

Show full code block

問問 Gemini 如何執行測試：

How do I run the tests?

在終端機中執行建議的指令，並查看輸出內容。

test_calendar.py

calendar-app-lab > test_calendar.py > ...

```
14
15 """Unit tests for the calendar module."""
16
17 import unittest
18 from calendar import number_to_roman
19
20 class TestNumberToRoman(unittest.TestCase):
21     """Test cases for the number_to_roman function."""
22
23     def test_simple_numbers(self):
24         """Tests conversion of simple, single-character Roman numerals"""
25         self.assertEqual(number_to_roman(1), "I")
26         self.assertEqual(number_to_roman(5), "V")
27         self.assertEqual(number_to_roman(10), "X")
28         self.assertEqual(number_to_roman(50), "L")
29         self.assertEqual(number_to_roman(100), "C")
30         self.assertEqual(number_to_roman(500), "D")
31         self.assertEqual(number_to_roman(1000), "M")
32
33     def test_additive_method(self):
34         """Tests the function's additive-only conversion logic."""
35         # The function uses additive notation, so 4 is IIII, not
36         self.assertEqual(number_to_roman(4), "IIII")
37         self.assertEqual(number_to_roman(9), "VIIII")
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

bash

Use 'gcloud config set project [PROJECT_ID]' to change to a different project.
student_01_8e405d66fe7d@cloudshell:~ (qwiklabs-gcp-04-6804c3b88fb8) \$ python3 test_calendar.py
python3: can't open file '/home/student_01_8e405d66fe7d/test_calendar.py': [Errno 2] No such file or directory
student_01_8e405d66fe7d@cloudshell:~ (qwiklabs-gcp-04-6804c3b88fb8) \$ cd /home/student_01_8e405d66fe7d/calendar-app-lab/
student_01_8e405d66fe7d@cloudshell:~/calendar-app-lab (qwiklabs-gcp-04-6804c3b88fb8) \$ python3 test_calendar.py
.....
Ran 7 tests in 0.001s
OK
student_01_8e405d66fe7d@cloudshell:~/calendar-app-lab (qwiklabs-gcp-04-6804c3b88fb8) \$

GEMINI CODE ASSIST

how do I run the tests?

Gemini

Thinking

Of course! Based on the `test_calendar.py` file you have, you can run the tests using Python's built-in `unittest` module.

You can execute the test file directly from your terminal.

1. First, make sure you are in the correct directory:

bash

cd /home/student_01_8e405d66fe7d/calendar-app-lab/

2. Then, run the test script using `python3`:

bash

python3 test_calendar.py

Run in terminal

When you run this command, the `unittest` framework will discover and run all the tests within the `TestNumberToRoman` class. If all tests pass, you will see a message indicating the number of tests run and an "OK" status. If any test fails, it will provide detailed information about the failure,

Prompts to try

Refactor the 'number_to_roman' function to be more efficient and support standard subtractive notation...

Add error handling to the Flask app in 'main.py' to gracefully handle invalid number inputs.

Ask Gemini or type '@'

1 context item

步驟 8 · 約 0 分鐘

8. 檢查是否有錯誤

開啟 `calendar.py` 檔案，然後在即時通訊視窗中傳送這項提示：

Are there any bugs in the conversion logic?

查看建議的變更，並在對話中接受：

GEMINI CODE ASSIST

🔍 + 🕒 ... ✕

Are there any bugs in the conversion logic?

◆ Gemini

⌵ Thinking

Of course. After reviewing the `number_to_roman` function, I've found that while its logic for converting positive integers is correct according to its documentation, there is a subtle bug in how it handles different types of invalid input.

The function behaves inconsistently:

- When given an invalid string like `"abc"`, it correctly allows a `ValueError` to be raised.
- However, when given a negative number like `-10`, it fails silently by returning an empty string.

This inconsistency can hide bugs in the code that uses this function. A more robust and reliable design would be to handle all invalid inputs consistently by raising an exception.

I'll provide a change to fix this bug by making the function raise a `ValueError` for negative numbers. I will also update the unit tests to reflect this improved behavior.

Here are the suggested changes:

⌵ calendar.py -0 +3

📄 🗖️ ✓

```
    A string representing the number in Roman numerals.
    """
    number = int(number)
```

Prompts to try

重新執行單元測試，驗證變更：

```
python3 test_calendar.py
```

如果看到錯誤訊息，可以選取錯誤並加入 Gemini 的背景資訊，協助排解及修正問題。

Ran 7 tests in 0.001s

OK

student_01_8e405d66fe7d@cloudshell:~/calendar-app-lab (qwiklabs-gcp-04-6804c3b88fb8)\$ python3 test_calendar.py

.....

Ran 7 tests in 0.001s

OK

student_01_8e405d66fe7d@cloudshell:~/calendar-app-lab (qwiklabs-gcp-04-6804c3b88fb8)\$ python3

○ student_01_8e405d66fe7d@cloudshell:~/calendar-app-lab (qwiklabs-gcp-04-6804c3b88fb8)\$ python3 test_calendar.py

步驟 9 · 約 0 分鐘

9. 重構 UI

在對話視窗中輸入下列提示：

Refactor UI to use bootstrap library

查看並接受變更：

GEMINI CODE ASSIST

Refactor UI to use bootstrap library

 Gemini

Thinking

Excellent idea! Using a modern CSS framework like Bootstrap will significantly improve the user interface of the application.

I will create the necessary HTML template files and style them with Bootstrap. Since the original templates weren't provided, I'll create new ones from scratch in a `templates` directory.

Additionally, to make the application more robust and user-friendly, I'll update the Flask `convert` endpoint in `main.py` to gracefully handle invalid user inputs. This will catch errors from the `number_to_roman` function (like non-numeric or negative inputs) and display a clear error message to the user using a new Bootstrap-styled error page.

Here are the changes:

main.py -5 +7

It takes the number `from` the form, converts it using the ``calendar`` and renders a page displaying the result.

Returns:
A rendered HTML template showing the original number and its

Accept changes

啟動應用程式，或重新載入頁面 (如果應用程式已在執行)。

如果應用程式尚未執行，請在終端機中啟動：

```
python3 main.py
```

重新載入頁面，然後查看變更。

Roman Numeral Converter

Enter a number:

Convert

Conversion Result

The Roman numeral for **25** is:

XXV

Convert another number

請嘗試傳送負數，驗證錯誤頁面。

Roman Numeral Converter

Enter a number:

Convert

Error

Input cannot be a negative number.

Try again

10. 重構後端

在對話視窗中輸入下列提示：

Store requests in memory and create a page to display conversion history. Add links on all pages to view the history.

在對話中查看並接受變更：

GEMINI CODE ASSIST

 +  ... ×

 student 4ad5e09a

Store requests in memory and create a page to display conversion history. Add links on all pages to view the history.

 Gemini

⬆ Thinking

Of course! As an experienced software engineering assistant, I can certainly help you add a conversion history feature to your application. This is a great idea to improve user experience.

I'll modify the application to store conversion requests in memory and display them on a new history page. I will also add navigation links to all pages so users can easily access this new feature.

Here are the required code changes:

1. Update the Main Application (`main.py`)

I'll add an in-memory list to store the history, create a new `/history` route to display it, and modify the `/convert` route to save each new conversion.

⬆ main.py -0 +9

```
import calendar

app = Flask(__name__)

history = []
```

Show full code block

2. Add Navigation to the Base Template (`base.html`)

在終端機中執行下列指令，啟動應用程式：


```
python3 main.py
```

存取應用程式並提交多項要求，然後查看記錄頁面。

Roman Numeral Converter

History

Roman Numeral Converter

Enter a number:

Convert!

查看轉換要求記錄。

Roman Numeral Converter

History

Conversion History

Number	Roman Numeral
100	C
25	XXV
25	XXV

Back to Converter

11. Gemini Code Assist 結論

您現在已瞭解 Gemini Code Assist 如何大幅簡化各種開發工作，包括解讀現有程式碼、生成說明文件和單元測試，以及重構 UI 和後端元件。這項工具可理解脈絡並提供相關建議，因此是開發人員的得力助手。

建議您進一步試用 Gemini Code Assist。嘗試不同的提示詞，使用自己的程式碼集探索其功能，並瞭解如何提升日常開發工作流程。與 Gemini Code Assist 互動越多，越能發掘其潛力，進而提升工作效率和程式碼品質。

步驟 12 · 約 0 分鐘

12. 恭喜！

恭喜，您已完成本程式碼研究室！

涵蓋內容：

- 使用 Gemini Code Assist 處理常見的開發人員工作

後續步驟：

- 更多實務課程即將推出！

清除所用資源

如要避免系統向您的 Google Cloud 帳戶收取本教學課程所用資源的費用，請刪除含有相關資源的專案，或者保留專案但刪除個別資源。

刪除專案

如要避免付費，最簡單的方法就是刪除您為了本教學課程所建立的專案。

©2024 Google LLC 保留所有權利。Google 和 Google 標誌是 Google LLC 的商標，所有其他公司和產品名稱可能是其關聯公司的商標。
